

Fiche de révision — Renforcement Programmation Orientée Objet (S7)

Méthodes clés pour traiter n'importe quel exercice de POO

1. Les 4 piliers de la POO

Pilier	Définition	Signal dans un énoncé
Encapsulation	cacher l'état interne, exposer un comportement via des méthodes publiques	« garantir l'intégrité des données », attributs privés + accesseurs
Héritage	une classe fille réutilise/spécialise une classe mère	« est un » (is-a), factorisation de code commun
Polymorphisme	une même interface se comporte différemment selon le type réel de l'objet	traiter une liste d'objets de types différents de façon uniforme
Abstraction	définir un contrat sans imposer l'implémentation	interfaces, classes abstraites, méthodes abstraites

2. Classe abstraite vs interface — comment choisir

Critère	Classe abstraite	Interface
État (attributs)	peut avoir des attributs et du code partagé	pas d'état (sauf constantes)
Héritage multiple	non (héritage simple)	oui, une classe peut implémenter plusieurs interfaces
Relation exprimée	« est un » avec comportement commun	« est capable de » (contrat de comportement)

Méthode d'exercice : si l'énoncé décrit plusieurs comportements indépendants qu'une classe pourrait cumuler (ex. « Volant » et « Nageur »), penser interfaces. Si l'énoncé décrit une hiérarchie avec du code/état réellement partagé, penser classe abstraite.

3. Redéfinition (overriding) vs surcharge (overloading)

- **Redéfinition** : une sous-classe fournit une nouvelle implémentation d'une méthode héritée avec exactement la même signature → liée au polymorphisme dynamique (résolue à l'exécution).
- **Surcharge** : plusieurs méthodes de même nom mais de signatures différentes (nombre/type de paramètres) dans la même classe → résolue à la compilation (polymorphisme statique).
- Piège fréquent : changer seulement le type de retour ne suffit pas à créer une surcharge valide dans la plupart des langages.

4. Composition vs héritage

- **Héritage** (« est un ») : couplage fort, la sous-classe hérite de toute l'implémentation, y compris ce qui ne convient pas toujours (risque de violer Liskov).
- **Composition** (« a un ») : un objet détient une référence vers un autre et lui délègue une partie du travail → plus flexible, recommandé quand la relation n'est pas clairement une spécialisation.

- Règle empirique à citer : « préférer la composition à l'héritage » quand il y a un doute, car elle réduit le couplage.

5. Design patterns comportementaux/créationnels utiles en TP

Patron	Idée en une phrase	Exemple d'usage
Factory Method	une méthode de création est redéfinie par les sous-classes pour instancier le bon type	créer différents types de véhicules selon un paramètre
Singleton	une seule instance accessible globalement	gestionnaire de configuration, connexion unique à une ressource
Strategy	encapsuler une famille d'algorithmes interchangeables derrière une interface commune	changer la méthode de tri ou de calcul de remise à l'exécution
Observer	des objets « abonnés » sont notifiés automatiquement d'un changement d'état du « sujet »	mise à jour d'une interface graphique quand une donnée change
Decorator	ajouter des responsabilités à un objet en l'enveloppant, sans modifier sa classe	ajouter des options à une commande sans multiplier les sous-classes

6. Gestion des exceptions

- Créer une exception personnalisée en héritant d'une classe d'exception existante quand l'erreur est spécifique au domaine métier.
- Ne capturer une exception que là où l'on peut réellement réagir ; sinon la laisser remonter (ou la relancer).
- Toujours libérer les ressources (fichiers, connexions) dans un bloc finally / try-with-resources, même en cas d'exception.

7. Généricité (types paramétrés)

Une classe ou méthode générique permet de manipuler différents types sans dupliquer le code, tout en gardant la vérification de type à la compilation (contrairement à un simple Object/Any qui nécessiterait des transtypages risqués). Exercice type : transformer une classe conteneur spécifique à un type (ex. ListeEntiers) en classe générique (Liste<T>).

8. Méthode générale pour un exercice « concevoir des classes »

- 1) Repérer les noms communs de l'énoncé → classes candidates.
- 2) Repérer les propriétés de chaque nom → attributs.
- 3) Repérer les verbes/actions → méthodes.
- 4) Chercher les relations « est un » → héritage ; « a un »/« possède » → composition/agrégation ; « peut faire » indépendamment du reste → interface.
- 5) Vérifier chaque relation avec SOLID (une classe = une responsabilité, éviter les dépendances concrètes inutiles).
- 6) Ne descendre dans la hiérarchie d'héritage que si le comportement diffère réellement, sinon un attribut ou une composition suffit.

9. Pièges classiques en examen

- Sur-utiliser l'héritage là où la composition serait plus appropriée (violations fréquentes du principe de Liskov).
- Oublier qu'une interface ne peut pas contenir d'état mutable.
- Confondre polymorphisme (exécution) et surcharge (compilation) dans une question de cours.
- Ne pas justifier le choix d'un design pattern par le problème réel de l'énoncé.